

Sistemas Distribuídos — Aula 17

Meios de Comunicação em Ambientes de Tempo Real e On-line · 08/10 · Prof. Leandro Borges

Comunicação em tempo real: requisitos

Em sistemas de tempo real, não basta a mensagem chegar corretamente — ela precisa chegar dentro de um prazo determinado. Uma resposta correta que chega atrasada pode ser tão inútil, ou pior, do que uma resposta incorreta que chega no prazo certo.

As restrições típicas desse tipo de sistema incluem latência previsível — o que importa não é apenas a latência média baixa, mas a baixa variação dessa latência — jitter controlado, ou seja, a variação de tempo entre pacotes consecutivos, e largura de banda garantida. Sistemas de tempo real frequentemente sacrificam a taxa de transferência máxima possível em troca de maior previsibilidade no comportamento da rede.

Tempo real rígido (hard) vs. flexível (soft)

No tempo real rígido (hard real-time), perder um prazo representa uma falha total do sistema, não apenas uma degradação de qualidade percebida pelo usuário. Exemplos incluem o controle de sistemas de frenagem de um veículo ou o controle de segurança de um processo industrial — nesses casos, chegar atrasado é equivalente a não funcionar.

No tempo real flexível (soft real-time), perder um prazo ocasionalmente degrada a qualidade da experiência do usuário, mas não representa uma falha catastrófica do sistema como um todo. Exemplos incluem uma videochamada com um quadro de vídeo atrasado, ou um jogo on-line enfrentando um pico pontual de latência de rede.

Protocolos de comunicação e streaming de dados

Na escolha de protocolo, o UDP costuma ser preferido ao TCP em cenários de tempo real: o TCP retransmite automaticamente pacotes perdidos, o que pode atrasar ainda mais uma comunicação que já é sensível a tempo. No UDP, perder um pacote antigo é geralmente preferível a atrasar a chegada dos pacotes mais recentes, que já tornaram o pacote perdido obsoleto de qualquer forma.

O streaming contínuo transmite os dados em um fluxo constante, processado em pequenos blocos (buffers) conforme eles chegam, em vez de esperar que a mensagem inteira seja recebida antes de começar a processá-la — isso permite, por exemplo, começar a exibir um vídeo antes que toda a transmissão tenha terminado de chegar ao destino.

Estudo de caso: videoconferência e jogos on-line

Sistemas de videoconferência priorizam continuidade sobre perfeição: preferem descartar um quadro de vídeo corrompido ou atrasado e seguir em frente com a transmissão, em vez de pausar tudo esperando por uma retransmissão — pequenas falhas visuais pontuais são consideradas mais toleráveis pelo usuário do que travamentos completos da chamada.

Jogos on-line multiplayer usam técnicas como a predição de movimento no lado do cliente: o próprio jogo "adivinha" qual será o próximo movimento de um personagem antes mesmo de receber a confirmação do servidor, e corrige silenciosamente a posição exibida na tela caso o servidor acabe discordando dessa previsão local. Essa técnica disfarça a latência real da rede aos olhos do jogador, tornando a experiência mais fluida do que ela tecnicamente é.

Referências

STANKOVIC, J. A. Misconceptions About Real-Time Computing. IEEE Computer.

TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. São Paulo: Pearson.